BACHELORARBEIT · VERSTÄNDLICH ERKLÄRT · RECHENWEGE

Wie rechnen die Modelle?

Dieses Heft rechnet die Formeln der Arbeit einmal mit echten Zahlen durch: die beiden Baselines, Ridge, Random Forest und XGBoost, dazu die Gütemaße und die Tests. Ein einziger Stadtteilmonat läuft dabei durch alle Modelle. Wo der Abgabe-Code eine Rechnung ausführt, steht dabei, in welcher Datei und Zeile und wie dieser Code aussieht.

Titel der Arbeit: „Vorhersage von Feuerwehreinsätzen auf Basis struktureller und sozioökonomischer Merkmale – ein Vergleich ausgewählter Machine-Learning-Methoden am Beispiel von Stadtteildaten“

Grundlage: abgegebene Fassung vom 07.10.2026 und der Abgabe-Code (Ordner Bachelorarbeit_LukasNiessen_[redacted])

Beispielzahlen: am 11.10.2026 nachgerechnet, mit den Paketversionen aus requirements.txt

Seitenangaben: „S.“ meint immer die Seite in der Arbeit

Markierung: ZUSATZ kennzeichnet Angaben, die nicht in der Arbeit stehen

TEIL 1

Was gerechnet wird und wo

> An der Arbeit sind fünf Modelle beteiligt: zwei Referenzmodelle ohne einstellbare Schrauben und drei Verfahren, deren Hyperparameter gesucht werden (S. 48). Dazu kommt Stufe 1 der Messlatte, die ganz ohne Merkmale auskommt. Jedes Modell ist am Ende eine Rechenvorschrift: Sie macht aus den zwölf Merkmalen eines Stadtteilmonats eine Zahl oder eine Einsatzart. Dieses Heft führt jede dieser Vorschriften einmal mit echten Zahlen vor.

Der gemeinsame Rechenweg

Im Mengenstrang gehen das Poisson-GLM und die drei Verfahren denselben Weg. Sie lesen die zwölf Merkmale einer Zeile, rechnen daraus die Einsätze je 1.000 Einwohner und kommen erst mit der Einwohnerzahl zur Anzahl der Einsätze (S. 51). Was zwischen Merkmalen und Rate passiert, unterscheidet die Modelle. Im Strukturstrang steht am Ende keine Zahl, sondern eine von vier Einsatzarten.

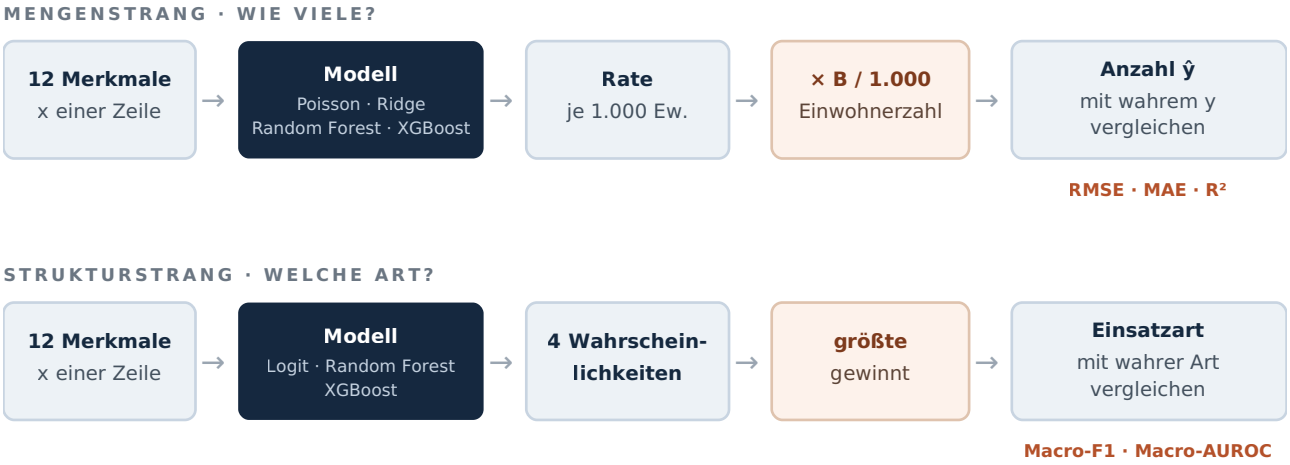


Abbildung 1: Der Rechenweg beider Stränge. Beim Poisson-GLM steckt die Multiplikation mit der Einwohnerzahl schon im Modell (Offset, Teil 3), bei den drei Verfahren folgt sie nach der Vorhersage (Teil 5). Stufe 1 der Messlatte nimmt diesen Weg nicht: Sie setzt ohne Merkmale den Mittelwert beziehungsweise die häufigste Klasse der Trainingszeilen ein (Teil 2).

Wo jede Rechnung steht

Rechnung	In der Arbeit	Im Abgabe-Code	Teil
Stufe 1 Menge: Gesamtmittelwert	ohne Nummer, S. 41	`vorpruefung/v1_baselines.py, Z. 167`	2
Stufe 1 Art: Mehrheitsklasse	ohne Nummer, S. 41	`vorpruefung/v1_baselines.py, Z. 238, 249–250`	2
Stufe 2 Menge: Poisson-GLM	Formeln 1, 2, 2a, S. 4–5	`vorpruefung/v1_baselines.py, Z. 95–108`	3
Stufe 2 Art: multinom. Logit	Formel 3, S. 6	`vorpruefung/v1_baselines.py, Z. 131–138`	4
Ridge Regression	Formeln 4, 4a, S. 7–8	`modelle/m02_menge.py, Z. 110`	5
Random Forest	keine Formel, S. 8	`modelle/m02_menge.py, Z. 111–112` `modelle/m03_struktur.py, Z. 86–89`	6
XGBoost	Formel 5, S. 9	`modelle/m02_menge.py, Z. 113–115` `modelle/m03_struktur.py, Z. 90–94`	7
Rate auf Anzahl umrechnen	ohne Nummer, S. 51	`modelle/m02_menge.py, Z. 219–231`	5
RMSE, MAE, R^2	in Worten, S. 10	`vorpruefung/v1_baselines.py, Z. 59–70` `modelle/m02_menge.py, Z. 268–270`	8
Macro-F1, Macro-AUROC	Formel 6, S. 10	`vorpruefung/v1_baselines.py, Z. 255–256, 268–287` `modelle/m03_struktur.py, Z. 260, 327–347`	9
Mittelung, Wilcoxon, d, Holm	in Worten, S. 11–12, 19	`modelle/m02_menge.py, Z. 437–454, 484–527` `modelle/trennschaerfe.py, Z. 44–53`	10
Kriminalitätsindex	Formel 7, S. 34	`prep/s1_daten.py, Z. 508–530`	A

Tabelle 1: Alle Rechnungen dieses Hefts mit ihrer Stelle in der Arbeit und im Code. Die Zeilennummern gelten für den Abgabe-Code.

Ein Stadtteilmonat für alle Rechnungen

DAS RECHENBEISPIEL

Bernal Heights im Oktober 2019. In Wiederholung 0 gehört Bernal Heights zu Fold 1 und ist dort Teststadtteil. Alle Modelle lernen also auf den 24 übrigen Entwicklungsstadtteilen (3.168 Zeilen, im Strukturstrang 3.167, weil dort ein Monat ohne Einsatz fehlt, S. 33) und sagen danach die 792 Zeilen der sechs Teststadtteile vorher, darunter diese eine. Tatsächlich gab es in diesem Monat 54 Einsätze bei 25.840 Einwohnern, also 2,09 Einsätze je 1.000 Einwohner. Die überwiegende Einsatzart war Fehlalarm.

Wer nachsehen will: Die Zeile steht in `data/processed/regression.parquet` mit `stadtteil = "Bernal Heights"` und `jahr_monat = 201910`.

Merkmal	Spalte in regression.parquet	Wert
Medianeinkommen	`median_haushaltseinkommen`	93.690 USD
Armutsquote	`armutsquote_pct`	0,0872
Akademikerquote	`akademikerquote_pct`	0,3115
Medianmiete	`median_miete`	1.525 USD
Leerstandsquote	`leerstandsquote_pct`	0,0655
Wohnbevölkerung, log.	`log_bevoelkerung`	10,1597 (= log(1 + 25.840))
Kriminalitätsindex, log.	`log_kriminalitaetsindex`	−0,4829 (Index 0,617)
Altbauanteil vor 1940	`anteil_altbau_vor_1940_pct`	0,6904
Wohngebäudeanteil	`anteil_wohngebaeude_pct`	0,9034
Risikogewerbeanteil	`anteil_risikogewerbe_pct`	0,0402
Saison, Sinus	`monat_sin`	−0,8660 (Oktober)
Saison, Kosinus	`monat_cos`	0,5000 (Oktober)

Tabelle 2: Die zwölf Merkmale der Beispielzeile. Anteile stehen als Dezimalzahl, 0,0872 heißt also 8,72 %. Der Sinus- und Kosinusterm des Oktobers sind sin(2π · 10/12) und cos(2π · 10/12) (S. 36).

Modell	Vorhersage ŷ	Fehler y − ŷ	Herkunft der Zahl	Teil
Stufe 1: Gesamtmittelwert	69,86	−15,86	nachgerechnet	2
Stufe 2: Poisson-GLM	55,74	−1,74	nachgerechnet	3
Ridge Regression	59,37	−5,37	menge_vorhersagen.parquet	5
Random Forest	53,84	+0,16	menge_vorhersagen.parquet	6
XGBoost	53,63	+0,37	menge_vorhersagen.parquet	7
Wahrer Wert	**54**			

Tabelle 3: Was die fünf Modelle des Mengenstrangs für die Beispielzeile vorhersagen. Die Baselines speichern keine Einzelvorhersagen, sondern nur ihre Gütemaße. Ihre Werte sind deshalb nachgerechnet. Dass die Nachrechnung stimmt, zeigt der RMSE des ganzen Folds: Er trifft den gespeicherten Wert 32,466 auf alle angegebenen Stellen.

Eine Zeile ist kein Befund. Dass Random Forest und XGBoost hier am nächsten liegen, sagt nichts über ihre Güte. Darüber entscheiden die Gütemaße über alle Testzeilen und alle Wiederholungen (Teile 8 und 10). Die Beispielzeile zeigt nur, wie gerechnet wird.

TEIL 2

Stufe 1 · Raten ohne Merkmale

> Die erste Stufe der Messlatte schaut sich kein einziges Merkmal an. Sie beantwortet nur eine Frage: Steckt in den Merkmalen überhaupt Information (S. 41)? Ein Modell, das Merkmale nutzt, sollte diese Stufe mindestens schlagen.

Mengenstrang: der Gesamtmittelwert

Stufe 1 sagt für jede Testzeile dieselbe Zahl vorher: den Mittelwert der Zielgröße über alle Trainingszeilen. Eine Formelnummer hat diese Rechnung nicht, die Arbeit beschreibt sie in Worten. Als Formel geschrieben:

$$\hat{y} = \frac{1}{n_{\text{Training}}} \sum_{i \in \text{Training}} y_i$$

Stufe 1 im Mengenstrang · in der Arbeit in Worten (S. 41)

In Fold 1 haben die 3.168 Trainingszeilen im Mittel 69,86 Einsätze. Diese 69,86 bekommt jede der 792 Testzeilen, auch Bernal Heights mit seinen 54. Auf der Rate ist es dasselbe mit 3,40 Einsätzen je 1.000 Einwohner.

```
163 for ziel, referenz in ((ZIELGROESSE, anzahl),
164                        (RATE, anzahl / bev * 1000)):
165     y = test[ziel].to_numpy()
166     for stufe, name, y_hat in (
167             (1, NULLMARKE, np.full(len(test), train[ziel].mean())),
168             (2, POISSON, referenz)):
```

vorpruefung/v1_baselines.py, Z. 163–168

Z. 167 `np.full` baut eine Liste, die so lang ist wie die Testmenge und überall denselben Wert trägt: `train[ziel].mean()`, den Mittelwert der Trainingszeilen.

Z. 163–164 Die Schleife läuft zweimal: einmal mit der Anzahl, einmal mit der Rate als `ziel`. Beide Mittelwerte entstehen so im selben Lauf.

Warum das Bestimmtheitsmaß negativ wird

Das Bestimmtheitsmaß vergleicht den Fehler eines Modells mit dem Fehler, den man macht, wenn man einfach den Mittelwert der *Testdaten* vorhersagt (Teil 8). Stufe 1 sagt aber den Mittelwert der *Trainingsdaten* vorher. Die sechs Teststadtteile von Fold 1 haben im Mittel 83,03 Einsätze, nicht 69,86. Stufe 1 setzt also am falschen Niveau an, und ihr Fehler wird größer als der Vergleichsfehler (S. 45). Rechnerisch ist ihre Fehlerquadratsumme genau um $792 \cdot (83{,}03 - 69{,}86)^2 \approx 137.500$ größer als die um den Testmittelwert ZUSATZ.

1	Fehlerquadrate von Stufe 1	$\Sigma\,(y - 69{,}86)^2$ über 792 Testzeilen	**2.643.107**
2	Fehlerquadrate um den Testmittelwert	$\Sigma\,(y - 83{,}03)^2$ über 792 Testzeilen	**2.505.607**
3	Bestimmtheitsmaß	$1 - 2.643.107 / 2.505.607 = 1 - 1{,}0549$	**−0,055**

Genau dieser Wert steht in `results/regression/baselines_folds.csv` in der Zeile für Wiederholung 0, Fold 1, Stufe 1. Über alle 50 Läufe gemittelt sind es −0,865 (Tabelle 6, S. 44).

Strukturstrang: die Mehrheitsklasse

Im Strukturstrang sagt Stufe 1 immer die Einsatzart vorher, die in den Trainingszeilen am häufigsten ist. In Fold 1 ist das Fehlalarm mit 2.506 von 3.167 Trainingszeilen. Alle 792 Testzeilen bekommen also „Fehlalarm“. 642 davon sind wirklich Fehlalarm-Monate, die Accuracy liegt damit bei 642/792 = 0,811.

Das Macro-F1 bleibt trotzdem niedrig. Für Fehlalarm liegt das F1 bei 0,895, für die drei anderen Klassen bei null, weil sie nie vorhergesagt werden. Der Durchschnitt über vier Klassen ist 0,895/4 = 0,224. Mehr als ein Viertel kann die Mehrheitsklasse nie erreichen, denn drei der vier Summanden sind immer null. Über alle Läufe sind es 0,221 (Tabelle 6, S. 44). Wie das F1 einer Klasse entsteht, rechnet Teil 9 vor.

```
238 haeufigste = y_tr.value_counts().idxmax()
248 for stufe, name, y_hat, au in (
249         (1, f"Mehrheitsklasse ({haeufigste})",
250          np.full(len(y_te), haeufigste), np.nan),
251         (2, LOGREG, logreg.predict(X_te), auroc)):
```

vorpruefung/v1_baselines.py, Z. 238–251, gekürzt

Z. 238 `value_counts()` zählt die Klassen der Trainingszeilen, `idxmax()` nimmt die häufigste.

Z. 249–250 Wie im Mengenstrang füllt `np.full` die ganze Testmenge mit diesem einen Wert. Eine Macro-AUROC gibt es für Stufe 1 nicht (`np.nan`), weil eine konstante Vorhersage keine Rangfolge hat.

TEIL 3

Stufe 2 · das Poisson-GLM

> Die eigentliche Messlatte im Mengenstrang ist ein Poisson-Modell mit Offset. Es gehört zu den verallgemeinerten linearen Modellen und hat keinen einzigen Hyperparameter. Seine Rechnung lässt sich vollständig von Hand nachvollziehen. Genau das macht dieser Teil.

Die Grundform: Formel 1

$$g\big(\mathbb{E}[y \mid \mathbf{x}]\big) = \mathbf{x}^{\top}\boldsymbol{\beta}$$

Formel 1 der Arbeit · allgemeine Form eines verallgemeinerten linearen Modells (S. 4)

$\mathbb{E}[y \mid \mathbf{x}]$ Der erwartete Wert der Zielgröße bei diesen Merkmalen, kurz μ. Hier: die erwartete Zahl der Einsätze.

$\mathbf{x}^{\top}\boldsymbol{\beta}$ Die gewichtete Summe: jedes Merkmal mal seinem Koeffizienten, alles addiert, dazu eine Konstante. Dieser Teil ist in jedem GLM gleich.

$g(\dots)$ Die Linkfunktion. Sie verbindet die gewichtete Summe mit dem Erwartungswert. Beim Poisson-Modell ist es der Logarithmus.

Der Logarithmus hat einen praktischen Grund: Die gewichtete Summe kann jeden Wert annehmen, auch einen negativen. Löst man nach μ auf, steht sie im Exponenten, und e hoch eine beliebige Zahl ist immer positiv. Negative Einsatzzahlen sind so ausgeschlossen (S. 5).

Formel 2 und 2a: Rate und Offset

$$\log(\mu/\mathrm{B}) = \mathbf{x}^{\top}\boldsymbol{\beta} \qquad \log(\mu) = \log(\mathrm{B}) + \mathbf{x}^{\top}\boldsymbol{\beta}$$

Formel 2 und 2a der Arbeit · Poisson-Regression für die Rate und mit Offset (S. 5)

Formel 2 setzt nicht die Einsatzzahl μ an, sondern die Rate μ/B, also die Einsätze je Einwohner. Formel 2a ist dieselbe Gleichung, nur umgestellt: Der Logarithmus der Einwohnerzahl B wird zur gewichteten Summe addiert, mit festem Gewicht eins. Das Modell darf an diesem Gewicht nichts verändern. Ein so vorgegebener Term heißt Offset (S. 5). Löst man nach μ auf, wird sichtbar, was er bewirkt:

$$\mu = \mathrm{B} \cdot \exp\left(\mathbf{x}^\top \boldsymbol{\beta}\right)$$

Formel 2a, nach μ aufgelöst

Die erwartete Einsatzzahl ist die Einwohnerzahl mal ein Faktor, der aus den Merkmalen kommt. Dieser Faktor ist die erwartete Zahl der Einsätze je Einwohner, mal 1.000 genommen die Rate je 1.000 Einwohner.

Durchgerechnet für Bernal Heights

In Fold 1 hat das Modell auf den 3.168 Trainingszeilen dreizehn Koeffizienten geschätzt: eine Konstante und einen je Merkmal. Für die Beispielzeile wird jedes Merkmal mit seinem Koeffizienten multipliziert. Die Summe dieser Beiträge ist die gewichtete Summe aus Formel 2a.

Merkmal	Wert x_j	Koeffizient β_j	Beitrag $\beta_j \cdot x_j$
Konstante β_0	1	−5,9152	−5,9152
Medianeinkommen	93.690	$-3{,}783 \cdot 10^{-7}$	−0,0354
Armutsquote	0,0872	2,8242	0,2463
Akademikerquote	0,3115	0,4765	0,1484
Medianmiete	1.525	$1{,}441 \cdot 10^{-4}$	0,2198
Leerstandsquote	0,0655	2,9283	0,1918
Wohnbevölkerung, log.	10,1597	−0,1169	−1,1877
Kriminalitätsindex, log.	−0,4829	0,2413	−0,1165
Altbauanteil vor 1940	0,6904	0,6021	0,4157
Wohngebäudeanteil	0,9034	−0,2731	−0,2467
Risikogewerbeanteil	0,0402	1,9948	0,0802
Saison, Sinus	−0,8660	−0,0318	0,0276
Saison, Kosinus	0,5000	0,0658	0,0329
Gewichtete Summe $\mathbf{x}^\top\boldsymbol{\beta}$			**−6,1390**

Tabelle 4: Die gewichtete Summe des Poisson-GLM für die Beispielzeile, Fold 1 der Wiederholung 0. Die Koeffizienten von Einkommen und Miete sind winzig, weil diese Merkmale in Dollar gemessen werden: Ein Dollar mehr ändert wenig. Gerechnet ist mit ungerundeten Werten.

1	Gewichtete Summe	Summe aus Tabelle 4	**$\mathbf{x}^\top\boldsymbol{\beta} = -6{,}1390$**
2	Offset dazu	log(25.840) = 10,1597	**log μ = 4,0207**
3	Zurück mit der e-Funktion	$e^{4{,}0207}$	**μ = 55,74**
4	Rate je 1.000 Einwohner	$55{,}74 / 25.840 \cdot 1.000 = e^{-6{,}1390} \cdot 1.000$	**2,157**

Das Modell erwartet also 55,74 Einsätze, tatsächlich waren es 54. Der Fehler beträgt −1,74.

WIE MAN EINEN KOEFFIZIENTEN LIEST

Weil die Summe im Exponenten steht, wirkt jeder Koeffizient als Faktor (S. 5). Ein Beispiel aus Tabelle 4: Steigt die Armutsquote um 0,10, also um zehn Prozentpunkte, wächst die Summe um 2,8242 · 0,10 = 0,2824. Die erwartete Rate wächst dann um den Faktor $e^{0{,}2824}$ = 1,33, also um 33 %, solange alle anderen Merkmale gleich bleiben ZUSATZ. Das beschreibt einen Zusammenhang in den Trainingsdaten von Fold 1 und keine Ursache. Die Arbeit ist prognoseorientiert und erhebt keinen kausalen Anspruch (S. 17).

Die Einwohnerzahl steckt zweimal in der Rechnung. Als Offset log(B) geht sie mit festem Gewicht eins ein und macht aus der Zahl eine Rate. Als Merkmal log(1 + B) bekommt sie einen geschätzten Koeffizienten (S. 36). In Fold 1 ist er negativ (−0,1169): Bei sonst gleichen Merkmalen hat ein größerer Stadtteil etwas weniger Einsätze je Einwohner. Dass das Merkmal log(1 + B) statt log(B) verwendet, legt `prep/s2_datensaetze.py` in Z. 239 fest. Bei 25.840 Einwohnern beträgt der Unterschied nur 0,00004 ZUSATZ.

Der Code dazu

```
 95  spalten = MERKMALE if merkmale is None else list(merkmale)
 96  X_tr = sm.add_constant(train[spalten].astype(float), has_constant="add")
 97  X_te = sm.add_constant(test[spalten].astype(float),  has_constant="add")
 98  y_tr = train[ZIELGROESSE].astype(float)
102  off_tr = np.log(train[EXPOSURE_ROH].astype(float))
103  off_te = np.log(test[EXPOSURE_ROH].astype(float))
106  modell = sm.GLM(y_tr, X_tr, family=sm.families.Poisson(),
107                  offset=off_tr).fit()
108  return np.asarray(modell.predict(X_te, offset=off_te))
```

vorpruefung/v1_baselines.py, Z. 95–108, gekürzt

Z. 95 `MERKMALE` setzt Z. 50 aus `PRAEDIKTOREN` und `SAISON` aus `prep/config.py` zusammen: dieselben zwölf Merkmale wie bei allen anderen Modellen.

Z. 96–97 `add_constant` hängt eine Spalte mit lauter Einsen an. Ihr Koeffizient ist die Konstante β_0 = −5,9152 aus Tabelle 4.

Z. 102–103 Der Offset log(B), getrennt für Trainings- und Testzeilen. `np.log` ist der natürliche Logarithmus.

Z. 106–107 `family=Poisson()` wählt die Poisson-Verteilung, deren Voreinstellung ist der Logarithmus als Link. `.fit()` schätzt die dreizehn Koeffizienten per Maximum Likelihood und ohne Strafterm (S. 6, S. 43).

Z. 108 `predict` mit `offset=off_te` rechnet genau die drei ersten Schritte oben: Summe, plus log(B), e hoch. Ohne den Offset käme statt der Einsatzzahl nur der Faktor je Einwohner heraus (S. 43).

Die Rate entsteht aus derselben Vorhersage, in Z. 164 mit `anzahl / bev * 1000`. Ein eigenes Ratenmodell gibt es nicht (S. 42).

Was „Maximum Likelihood“ hier heißt ZUSATZ: `.fit()` sucht die dreizehn Zahlen, unter denen die beobachteten Einsatzzahlen der Trainingszeilen nach der Poisson-Verteilung am wahrscheinlichsten sind. Das geschieht schrittweise. In Fold 1 war statsmodels nach 6 Schritten fertig.

TEIL 4

Stufe 2 · das multinomiale Logit

> Im Strukturstrang ist die Messlatte ein multinomiales Logit. Es rechnet für jede der vier Einsatzarten eine gewichtete Summe und macht daraus vier Wahrscheinlichkeiten. Vorhergesagt wird die Einsatzart mit der größten.

Woher die Einsatzart kommt

Die Zielgröße ist keine gemessene Zahl. Sie wird aus den Einsätzen gebildet: Die überwiegende Einsatzart ist die Einsatzgruppe mit dem größten Anteil (S. 36). In Bernal Heights gab es im Oktober 2019 9 Brand-, 11 Rettungs-, 13 Technische-Hilfe- und 21 Fehlalarm-Einsätze. Fehlalarm hat mit 21/54 = 38,9 % den größten Anteil und ist damit die Klasse dieser Zeile.

Formel 3

$$\log \frac{\Pr(y = k \mid \mathbf{x})}{\Pr(y = K \mid \mathbf{x})} = \mathbf{x}^{\top} \beta_k, \qquad k \in \{1, \ldots, K-1\}$$

Formel 3 der Arbeit · multinomiale logistische Regression gegen eine Bezugsklasse (S. 6)

k, K k ist eine der Einsatzarten, etwa Brand. K ist die Bezugsklasse, mit der alle anderen verglichen werden. Welche das ist, ist Konvention. Für das Beispiel dient Fehlalarm als Bezug, weil es die häufigste Klasse ist.

Bruch Das Chancenverhältnis: wie viel mal wahrscheinlicher k ist als die Bezugsklasse. Links steht sein Logarithmus.

β_k Jede Einsatzart hat ihren eigenen Satz Koeffizienten, also ihre eigene gewichtete Summe.

SO RECHNET SCIKIT-LEARN

Der Code schätzt nicht drei, sondern vier Koeffizientensätze, einen je Einsatzart, und rechnet die Wahrscheinlichkeiten mit der Softmax-Funktion aus ZUSATZ:

$$\Pr(y = k \mid \mathbf{x}) = \frac{\exp\left(\mathbf{x}^{\top} \beta_k\right)}{\sum_{l=1}^{K} \exp\left(\mathbf{x}^{\top} \beta_l\right)}$$

Das ist dasselbe Modell. Teilt man zwei dieser Wahrscheinlichkeiten durcheinander, kürzt sich der Nenner, und übrig bleibt e hoch die Differenz der beiden Summen. Genau das ist Formel 3. Die Arbeit sagt dazu, das logarithmierte Chancenverhältnis zweier Klassen sei die Differenz ihrer Koeffizientenvektoren (S. 6).

Durchgerechnet für Bernal Heights

Das Logit standardisiert die Merkmale zuerst: Von jedem Merkmal wird sein Mittelwert in den Trainingszeilen abgezogen, das Ergebnis wird durch die Standardabweichung geteilt. Für die standardisierte Beispielzeile ergeben sich in Fold 1 diese vier gewichteten Summen:

Einsatzart	Summe $x^\top \beta_k$	Abstand zu Fehlalarm	Chance gegen Fehlalarm	Wahrscheinlichkeit
Brand	−1,311	−2,439	$e^{-2,439} = 0,087$	4,6 %
Rettung	−0,449	−1,577	$e^{-1,577} = 0,207$	10,9 %
Technische Hilfe	0,633	−0,495	$e^{-0,495} = 0,610$	32,0 %
Fehlalarm	1,128	0,000	$e^{0,000} = 1,000$	52,5 %
Summe			**1,903**	**100,0 %**

Tabelle 5: Vom Rohwert zur Wahrscheinlichkeit. Die Spalte „Abstand zu Fehlalarm“ ist die linke Seite von Formel 3 mit Fehlalarm als Bezugsklasse.

Die vier Chancen summieren sich zu 1,903. Weil alle vier Wahrscheinlichkeiten zusammen eins ergeben, ist Pr(Fehlalarm) = 1/1,903 = 0,525, und jede andere Wahrscheinlichkeit ist ihre Chance mal 0,525, etwa für Brand 0,087 · 0,525 = 0,046. Vorhergesagt wird die größte: Fehlalarm. Das stimmt hier.

Ausgeglichene Klassengewichte

Ohne Ausgleich würde das Logit vor allem Fehlalarm lernen, weil diese Klasse vier von fünf Zeilen stellt. `class_weight="balanced"` gibt deshalb jeder Trainingszeile ein Gewicht, das umgekehrt proportional zur Häufigkeit ihrer Klasse ist. Jede der vier Klassen wiegt dann insgesamt gleich viel (S. 42, S. 54). scikit-learn rechnet dafür $n / (K \cdot n_k)$, mit n Trainingszeilen, K = 4 Klassen und n_k Zeilen der Klasse k ZUSATZ.

Klasse	Trainingszeilen n_k	Gewicht je Zeile	Gewicht der Klasse insgesamt
Brand	20	3.167 / (4 · 20) = 39,59	791,75
Rettung	131	3.167 / (4 · 131) = 6,04	791,75
Technische Hilfe	510	3.167 / (4 · 510) = 1,55	791,75
Fehlalarm	2.506	3.167 / (4 · 2.506) = 0,32	791,75

Tabelle 6: Die Klassengewichte in Fold 1. Eine Brand-Zeile zählt beim Lernen so viel wie 125 Fehlalarm-Zeilen, und jede Klasse kommt insgesamt auf 3.167/4 = 791,75. Die Daten selbst bleiben unverändert.

Der Preis steht in der Arbeit: Die Accuracy des Logits liegt mit 0,590 unter der der Mehrheitsklasse mit 0,795, weil die Gewichte das Modell von der häufigsten Klasse weg zu den seltenen lenken (Tabelle 6, S. 44, und S. 45).

Der Code dazu

```python
spalten = MERKMALE if merkmale is None else list(merkmale)
# unpenalisiert (C = inf); ausgeglichene Klassengewichte statt Resampling:
# keine duplizierte und keine geloeschte Zeile
return make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=2000, C=np.inf, class_weight="balanced")
).fit(train[spalten].astype(float), train[ZIELKLASSE])
```

vorpruefung/v1_baselines.py, Z. 131–138

Z. 136 `StandardScaler` standardisiert die zwölf Merkmale, angepasst nur auf den Trainingszeilen.

Z. 137 `C` ist in scikit-learn der Kehrwert der Strafstärke. `C=np.inf` heißt deshalb: kein Strafterm (S. 6). `class_weight` setzt die Gewichte aus Tabelle 6.

Z. 138 `.fit` schätzt die Koeffizienten aus Merkmalen und überwiegender Einsatzart der Trainingszeilen.

Z. 245, 251 `predict_proba` liefert die vier Wahrscheinlichkeiten aus Tabelle 5 für die AUROC, `predict` die Klasse mit der größten.

TEIL 5

Ridge Regression · die gebremste Gerade

> Ridge ist das erste der drei Vergleichsverfahren. Es rechnet wie eine gewöhnliche lineare Regression eine gewichtete Summe der Merkmale, bremst aber die Koeffizienten mit einem Strafterm. Wie stark gebremst wird, legt der Hyperparameter λ fest, und nur ihn sucht die Hyperparametersuche.

Formel 4

$$\hat{\beta}^{\text{Ridge}} = \arg\min_{\beta} \left\{ \sum_{i=1}^{n} \left(y_i - x_i^{\top}\beta\right)^2 + \lambda \sum_{j=1}^{q} \beta_j^2 \right\}$$

Formel 4 der Arbeit · Zielfunktion der Ridge Regression (S. 7)

Σ_i (…)² Die Summe der Fehlerquadrate über alle Trainingszeilen: wie schlecht die Gerade zu den Daten passt.

$\lambda \Sigma_j \beta_j^2$ Der Strafterm: Jeder Koeffizient, der weit von null weg ist, macht die Summe größer.

arg min Gesucht sind die Koeffizienten, bei denen die Summe aus beidem am kleinsten ist.

λ Das Strafgewicht. Bei $\lambda = 0$ entsteht die gewöhnliche Kleinste-Quadrate-Schätzung, mit wachsendem λ rücken die Koeffizienten gegen null (S. 7). In scikit-learn heißt λ `alpha`.

Formel 4a an einem Mini-Beispiel

$$\hat{\beta}^{\text{Ridge}} = \left(X^{\top}X + \lambda I\right)^{-1} X^{\top} y$$

Formel 4a der Arbeit · geschlossene Lösung der Ridge Regression (S. 8)

Formel 4a sagt, dass sich die beste Lösung direkt ausrechnen lässt, ohne etwas auszuprobieren. Mit nur einem zentrierten Merkmal schrumpft die Matrizenrechnung auf einen einzigen Bruch ZUSATZ:

$$\hat{\beta} = \frac{\sum_i x_i y_i}{\sum_i x_i^2 + \lambda}$$

Formel 4a mit einem einzigen, zentrierten Merkmal

Ein Beispiel mit drei Zeilen: x = (−1; 0; 1) und y = (−2; 0; 2). Beide sind schon zentriert, ihr Mittelwert ist null. Dann ist $\Sigma\, x_i y_i = 2 + 0 + 2 = 4$ und $\Sigma\, x_i^2 = 2$.

Strafgewicht λ	Koeffizient β̂	Fehlerquadrate	Strafe λβ̂²	Summe nach Formel 4
0 (gewöhnliche Rechnung)	4 / 2 = 2	0	0	0
2	4 / (2 + 2) = 1	2	2	4
6	4 / (2 + 6) = 0,5	4,5	1,5	6

Tabelle 7: Je größer λ, desto kleiner der Koeffizient. Probe für λ = 2: Mit β = 2 wäre die Summe $0 + 2 \cdot 2^2 = 8$, mit β = 0,5 wäre sie $4{,}5 + 2 \cdot 0{,}5^2 = 5$. Der Wert aus der Formel, β = 1, liefert mit 4 tatsächlich das Minimum ZUSATZ.

λ steht im Nenner neben der Summe der Merkmalsquadrate. Daraus lässt sich ablesen, wie stark λ in der Arbeit wirkt. Im Code sind die zwölf Merkmale standardisiert, und dann ist jeder Diagonaleintrag von $\mathbf{X}^\top\mathbf{X}$, das Gegenstück zu $\Sigma\, x_i^2$, genau so groß wie die Zahl der Trainingszeilen: 3.168. Die Suche hat in Fold 1 λ = 1.427,6 gewählt (`results/regression/tuning.csv`). Der Strafterm erhöht jeden Diagonaleintrag also um 45 % ZUSATZ. Die Länge des Koeffizientenvektors schrumpft dadurch von 0,295 bei λ = 0 auf 0,203, um 31 % ZUSATZ. Formel 4a mit diesen Zahlen liefert dieselben zwölf Koeffizienten wie scikit-learn, bis auf eine Abweichung in der 17. Nachkommastelle ZUSATZ.

Durchgerechnet für Bernal Heights

Ridge rechnet in vier Schritten, die im Code als eine Kette hintereinander stehen. Angepasst wird nicht auf der Anzahl, sondern auf der Rate, und auch die nur in der Form log(1 + Rate) (S. 51–52):

$$z_j = \frac{x_j - \bar{x}_j}{s_j} \qquad \hat{y} = \left(\exp\left(\beta_0 + \sum_j \beta_j z_j\right) - 1\right) \cdot \frac{B}{1000}$$

Standardisieren, gewichtet summieren, zurückrechnen, mit der Einwohnerzahl hochrechnen · Code: m02_menge.py, Z. 110 und 231

Merkmal	Wert x_j	Mittel $\bar{x}_j$	Std.-Abw. s_j	z_j	β_j	$\beta_j \cdot z_j$
Medianeinkommen	93.690	122.986	47.427	−0,618	0,0039	−0,0024
Armutsquote	0,0872	0,1037	0,0612	−0,269	0,0764	−0,0205
Akademikerquote	0,3115	0,3605	0,0810	−0,605	0,0520	−0,0314
Medianmiete	1.525	2.035	575	−0,887	0,0075	−0,0067
Leerstandsquote	0,0655	0,0839	0,0401	−0,458	0,0892	−0,0408
Wohnbevölkerung, log.	10,1597	9,7055	0,8046	0,565	−0,0698	−0,0394
Kriminalitätsindex, log.	−0,4829	−0,3397	0,7649	−0,187	0,0562	−0,0105
Altbauanteil vor 1940	0,6904	0,7082	0,1830	−0,097	0,0380	−0,0037
Wohngebäudeanteil	0,9034	0,7893	0,1821	0,627	−0,0706	−0,0442
Risikogewerbeanteil	0,0402	0,0481	0,0515	−0,153	0,0968	−0,0148
Saison, Sinus	−0,8660	0	0,7071	−1,225	−0,0196	0,0240
Saison, Kosinus	0,5000	0	0,7071	0,707	0,0225	0,0159
Summe Σ β_j z_j						**−0,1745**

Tabelle 8: Die Ridge-Rechnung für die Beispielzeile, Fold 1. Mittel und Standardabweichung stammen aus den 3.168 Trainingszeilen. Weil die Merkmale standardisiert sind, lassen sich die Koeffizienten untereinander vergleichen.

1	Achsenabschnitt	Mittel von log(1 + Rate) der Trainingszeilen	**β_0 = 1,3677**
2	Gewichtete Summe	1,3677 − 0,1745	**1,1932**
3	Zurückrechnen	$e^{1,1932} - 1$	**2,2976 je 1.000**
4	Hochrechnen	2,2976 · 25.840 / 1.000	**59,37**

Das Ergebnis 59,37 steht so in `results/regression/menge_vorhersagen.parquet` (Wiederholung 0, Fold 1, Ridge, `anzahl_einsaetze`). Der Achsenabschnitt ist genau der Mittelwert von log(1 + Rate) über die Trainingszeilen: Bei zentrierten Merkmalen schätzt Ridge ihn so und bestraft ihn nicht (S. 8).

Warum Ridge im Mittel zu niedrig liegt: Die Rückrechnung mit e··· − 1 holt den Logarithmus zurück, aber nicht den Mittelwert. Der Mittelwert der logarithmierten Werte liegt unter dem Logarithmus ihres Mittelwerts. Ridge unterschätzt deshalb im Mittel um 7,1 Einsätze. Die Arbeit korrigiert das bewusst nicht (S. 52).

Der Code dazu

```
110  if name == "ridge": return
     make_pipeline(StandardScaler(),TransformedTargetRegressor(regressor=Ridge(),func=np.log1p,
     inverse_func=np.expm1))
```

modelle/m02_menge.py, Z. 110

```
219  auf_rate = ziel == ZIELGROESSE
220  y_tr = train[RATE if auf_rate else ziel].astype(float)
221  zurueck = (test[EXPOSURE_ROH].astype(float).to_numpy() / 1000.0
222             if auf_rate else 1.0)
224  modell = verfahren(name).set_params(**parameter)
227  modell.fit(X_tr, y_tr)
231  y_hat = modell.predict(X_te) * zurueck
```

modelle/m02_menge.py, Z. 219–231, gekürzt

Z. 110 Die Kette in einer Zeile: `StandardScaler` standardisiert, `TransformedTargetRegressor` rechnet die Zielgröße mit `np.log1p` auf log(1 + y) um und die Vorhersage mit `np.expm1` zurück, `Ridge()` ist Formel 4. Weil alles in der Pipeline steht, sehen Standardisierung und Umrechnung je Fold nur die Trainingszeilen (S. 53).

Z. 219–222 Auch wenn die Anzahl das Ziel ist, wird auf der Rate angepasst. `zurueck` ist dann die Einwohnerzahl durch 1.000, bei der Rate selbst einfach 1.

Z. 224 Baut das Modell mit dem gesuchten `alpha`, also λ. Dieselbe Zeile gilt für alle drei Verfahren.

Z. 231 Erst hier kommt die Einwohnerzahl ins Spiel: Vorhersage der Rate mal `zurueck`.

Gesucht wird λ in `tune()`, Z. 155–184, mit 100 Zufallsziehungen zwischen 10^{-5} und 10^{5} auf logarithmischer Skala (`config_modelle.py`, Z. 34) und einer inneren, nach Stadtteilen gruppierten Kreuzvalidierung über vier Folds (S. 56).

TEIL 6

Random Forest · der Durchschnitt vieler Bäume

> Für den Random Forest steht in der Arbeit keine Formel. Er besteht aus Hunderten Entscheidungsbäumen, die jeder für sich eine Vorhersage machen. Am Ende wird gemittelt. Beides lässt sich an der Beispielzeile genau verfolgen.

Ein einzelner Baum

Ein Entscheidungsbaum stellt eine Folge von Ja-Nein-Fragen an die Merkmale. Jede Zeile landet am Ende in genau einem Endknoten. Dort steht der Mittelwert der Trainingszeilen, die beim Lernen ebenfalls dort gelandet sind (S. 50). Weil der Baum nur gelernte Mittelwerte ausgibt, kann er nie eine negative Rate vorhersagen (S. 53).

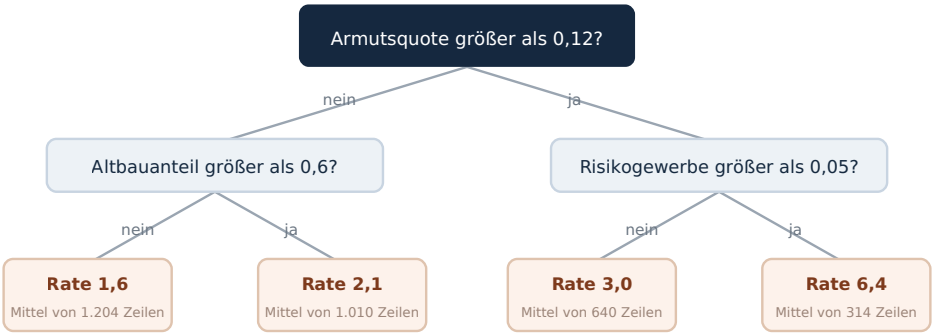


Abbildung 2: Ein Baum mit zwei Ebenen. Fragen, Schwellen und Werte sind erfunden und zeigen nur das Prinzip ZUSATZ. Die echten Bäume in Fold 1 sind im Mittel 12,9 Ebenen tief und haben rund 82 Endknoten ZUSATZ.

Der Wald

Die Arbeit beschreibt den letzten Schritt in Worten: Viele Bäume wachsen auf Bootstrap-Stichproben, ihre Vorhersagen werden gemittelt (S. 8). Als Formel geschrieben:

$$\hat{y}(\mathbf{x}) = \frac{1}{R} \sum_{r=1}^{R} T_r(\mathbf{x})$$

Mittelung über R Bäume · in der Arbeit in Worten (S. 8)

$T_r(x)$ Die Vorhersage des r-ten Baums für die Merkmale x, also der Wert des Endknotens, in dem die Zeile landet. R ist die Zahl der Bäume, ein gesuchter Hyperparameter.

Damit die Bäume nicht alle gleich aussehen, kommen zwei Zufallsquellen hinzu. Jeder Baum lernt auf einer Bootstrap-Stichprobe, einer Ziehung mit Zurücklegen aus den Trainingszeilen. Und an jedem Knoten darf er nur unter einer zufälligen Auswahl der Merkmale wählen (S. 8). In Fold 1 hat die Suche diese Einstellungen gefunden:

Hyperparameter	Code	Wert in Fold 1
Zahl der Bäume R	`n_estimators`	898
Merkmale je Split	`max_features`	Wurzel aus 12, also 3 von 12
Beobachtungen je Blatt, mindestens	`min_samples_leaf`	17
Baumtiefe, höchstens	`max_depth`	32

Tabelle 9: Die Einstellungen des Random Forest in Fold 1 (`results/regression/tuning.csv`). Die Tiefengrenze greift nicht: Mit mindestens 17 Zeilen je Blatt enden die Bäume schon nach 9 bis 18 Ebenen ZUSATZ.

Durchgerechnet für Bernal Heights

Jeder der 898 Bäume schickt die Beispielzeile durch seine Fragen und liefert eine Rate. Die ersten sechs Bäume sagen 3,50; 1,62; 2,00; 1,41; 1,55; 1,97 vorher. Über alle Bäume reicht die Spanne von 1,19 bis 8,57: Einzeln liegen die Bäume weit auseinander, gemittelt gleicht sich das aus.

Ihr Mittelwert über alle 898 Bäume ist **2,0837** Einsätze je 1.000 Einwohner. Mal 25.840/1.000 ergibt das **53,84** Einsätze.

Das ist auf alle Stellen der Wert in `menge_vorhersagen.parquet`. Die einzelnen Baumwerte stammen aus einer Nachrechnung mit denselben Einstellungen und demselben Startwert 42 ZUSATZ.

Wie ein Baum seine Fragen wählt

An jedem Knoten probiert der Baum Merkmale und Schwellen durch. Er nimmt die Teilung, nach der die Trainingszeilen in den beiden Teilen am besten zu ihrem jeweiligen Mittelwert passen. „Am besten“ misst hier nicht der quadratische Fehler, sondern die Poisson-Devianz (S. 52). Sie misst die Abweichung relativ zum erwarteten Niveau (S. 52). Weil der Random Forest auf der Rate angepasst wird, ist dieses Niveau die erwartete Rate: Derselbe Fehler wiegt bei niedriger Rate mehr als bei hoher ZUSATZ. Als Formel ZUSATZ:

$$D = 2 \sum_i \left[y_i \log \frac{y_i}{\hat{y}_i} - \left(y_i - \hat{y}_i \right) \right]$$

Poisson-Devianz · Zusatz

Ein Zahlenbeispiel ZUSATZ: Liegt die vorhergesagte Rate bei 1 und die wahre bei 3, kostet das 2,592. Liegt sie bei 8 und die wahre bei 10, kostet es nur 0,463. Der quadratische Fehler wäre in beiden Fällen $2^2 = 4$.

Der Code dazu

```
111 if name == "random_forest":
112     return RandomForestRegressor(random_state=RANDOM_STATE, n_jobs=n_jobs, criterion="poisson")
```

modelle/m02_menge.py, Z. 111–112

Z. 112 `RandomForestRegressor` mit `criterion="poisson"` für die Poisson-Devianz. `random_state` legt den Zufall fest, `n_jobs` rechnet auf einem Kern (S. 53).

Im Strukturstrang baut `m03_struktur.py` in Z. 86–89 einen `RandomForestClassifier` mit `class_weight="balanced"`, also denselben Klassengewichten wie das Logit. Im Endknoten steht dann statt einer Rate der Anteil jeder Einsatzart, gewichtet mit den Klassengewichten. Der Wald mittelt diese Anteile über alle Bäume und sagt die Einsatzart mit dem größten Mittel vorher ZUSATZ. Die Suchräume stehen in `config_modelle.py`, Z. 36–44, und sind in beiden Strängen gleich (S. 56).

TEIL 7

XGBoost · Bäume, die nachbessern

> XGBoost baut seine Bäume nicht nebeneinander, sondern nacheinander. Jeder neue Baum korrigiert, was die Summe der bisherigen noch falsch vorhersagt (S. 8). Formel 5 legt fest, was beim Bauen jedes Baums optimiert wird.

Die Vorhersage ist eine Summe

Am Ende ist die Vorhersage von XGBoost ein Startwert plus die Beiträge aller Bäume. Mit der Tweedie-Verlustfunktion rechnet XGBoost dabei wie das Poisson-Modell mit dem Logarithmus als Link (S. 53). Die Summe steht deshalb im Exponenten ZUSATZ:

$$\hat{y} = \exp\left(\log b + \sum_{r=1}^{R} f_r(\mathbf{x})\right)$$

Vorhersage von XGBoost mit Tweedie-Verlust · Zusatz

- b Der Startwert. XGBoost setzt ihn aus den Trainingsdaten, hier ist es die mittlere Rate der Trainingszeilen, 3,4035 ZUSATZ.
- $f_r(x)$ Der Wert des Endknotens, in dem die Zeile im r-ten Baum landet. Er ist schon mit der Schrittweite multipliziert ZUSATZ.
- R Die Zahl der Bäume, in Fold 1 sind es 406 (`results/regression/tuning.csv`).

Durchgerechnet für Bernal Heights

Die Tabelle verfolgt die Summe für die Beispielzeile, während die Bäume nacheinander dazukommen.

nach Baum	Summe auf der Log-Skala	Rate e^{Summe} je 1.000	Anzahl · B / 1.000
Start: log b	1,2248	3,403	87,95
1	1,1813	3,259	84,20
2	1,1402	3,127	80,81
3	1,1012	3,008	77,72
10	0,8816	2,415	62,40
50	0,6944	2,002	51,74
100	0,7276	2,070	53,49
200	0,7333	2,082	53,80
406	0,7333	2,082	53,80

Tabelle 10: Die Summe der Beispielzeile nach 0, 1, 2, 3, 10, 50, 100, 200 und allen 406 Bäumen, Fold 1. Am 11.10.2026 nachgerechnet ZUSATZ.

Der erste Baum zieht die Summe um 0,0435 nach unten. Genau diesen Wert trägt der Endknoten, in dem die Zeile im ersten Baum landet. Danach werden die Schritte kleiner, nach etwa 200 Bäumen ändert sich kaum noch etwas. Die Rate 2,082 mal 25.840/1.000 ergibt 53,80 Einsätze.

NACHGERECHNET, ABER NICHT AUF DIE STELLE GENAU

Tabelle 10 stammt aus einer Nachrechnung mit denselben Paketversionen, Daten und Parametern, aber auf einem anderen Rechner. Sie kommt auf 53,80, der gespeicherte Lauf auf 53,63 ZUSATZ. Bei Ridge stimmt jede einzelne Vorhersage des Folds überein, beim Poisson-GLM der gespeicherte RMSE, beim Random Forest jede Vorhersage bis auf weniger als 0,05 Einsätze. Bei XGBoost weicht die Nachrechnung im Fold um bis zu 17,9 Einsätze ab ZUSATZ. Dass XGBoost schon bei wechselnder Kernzahl eine andere Vorhersage liefert, beschreibt die Arbeit (S. 55). Für den Abgleich mit der Arbeit gilt deshalb der gespeicherte Wert 53,63.

Formel 5: was beim Bauen optimiert wird

$$L = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{r} \Omega(f_r) \quad \text{mit} \quad \Omega(f_r) = \gamma J_r + \tfrac{1}{2}\lambda \|w_r\|^2$$

Formel 5 der Arbeit · regularisierte Zielfunktion des Gradient Boosting (S. 9)

- $l(y_i, \hat{y}_i)$ Der Verlust einer Zeile: wie weit die Vorhersage danebenliegt. Hier ist es die Tweedie-Devianz (S. 52).
- $\Omega(f_r)$ Die Strafe für die Komplexität des r-ten Baums.
- J_r, γ Die Zahl der Endknoten und ihr Preis. In der Arbeit bleibt γ auf null, die Zahl der Endknoten kostet also nichts (S. 9).
- w_r, λ Die Werte in den Endknoten und das Strafgewicht dafür. Nur λ wird gesucht, in Fold 1 ergab sich λ = 189,4.

Ein Rechenbeispiel für den Strafterm ZUSATZ: Ein Baum mit drei Endknoten und den Werten 0,4; −0,2 und 0,1 kostet bei $\lambda = 2$ und $\gamma = 0$

$$\Omega = 0 \cdot 3 + ½ \cdot 2 \cdot (0{,}4^2 + (-0{,}2)^2 + 0{,}1^2) = 0{,}16 + 0{,}04 + 0{,}01 = 0{,}21.$$

Mit $\gamma = 1$ kämen noch $1 \cdot 3 = 3$ hinzu. Weil die Arbeit γ auf null lässt, kosten nur große Endknotenwerte etwas. Formel 5 hat damit denselben Bauplan wie Formel 4: links die Passung, rechts die Strafe.

WIE λ DIE ENDKNOTENWERTE KLEIN HÄLT ZUSATZ

Chen und Guestrin leiten aus Formel 5 den besten Wert eines Endknotens her (Chen & Guestrin 2016, S. 787):

$$w^* = -\frac{\sum_{i \in \text{Endknoten}} g_i}{\sum_{i \in \text{Endknoten}} h_i + \lambda}$$

g und h sind die erste und die zweite Ableitung des Verlusts jeder Zeile im Endknoten: g sagt, wie stark der Verlust steigt, wenn die Vorhersage steigt, h, wie stark er dabei gekrümmt ist. Verschoben wird gegen g, daher das Minuszeichen. λ steht wie im Ridge-Mini-Beispiel im Nenner. Je größer λ, desto kleiner der Wert, den ein Endknoten bekommt.

Zwei weitere Stellgrößen gehören dazu. Die **Schrittweite** multipliziert die Werte jedes neuen Baums, in Fold 1 mit 0,0744, damit kein einzelner Baum die Vorhersage festlegt (S. 9). Der **Varianzexponent** p der Tweedie-Devianz legt fest, wie die Streuung mit dem Niveau wächst: Die Varianz ist proportional zu μ^p, mit p = 1 für Poisson und p = 2 für die Gamma-Verteilung (Kommentar in `config_modelle.py`, Z. 54) ZUSATZ. In Fold 1 wählte die Suche p = 1,726 (tuning.csv).

Der Code dazu

```python
if name == "xgboost":
    from xgboost import XGBRegressor
    return XGBRegressor(random_state=RANDOM_STATE, n_jobs=n_jobs,objective="reg:tweedie")
```

modelle/m02_menge.py, Z. 113–115

```python
"xgboost": {
    "n_estimators":     ("int", 200, 1000),
    "learning_rate":    ("loguniform", 0.01, 0.3),
    "max_depth":        ("int", 1, 14),
    "subsample":        ("uniform", 0.6, 1.0),
    "colsample_bytree": ("uniform", 0.6, 1.0),
    "reg_lambda":       ("loguniform", 1e-4, 1e4),
    # Tweedie-Exponent p (Var = mu^p; 1 Poisson, 2 Gamma), getunt statt fest
    # 1,5. Untergrenze 1,01 schliesst die Loesung der Poisson-Baseline ein
    # (reg:tweedie verlangt 1 < p < 2). Nur Regression; m03 entfernt ihn.
    "tweedie_variance_power": ("uniform", 1.01, 1.9),
```

modelle/config_modelle.py, Z. 45–57

m02, Z. 115 `XGBRegressor` mit `objective="reg:tweedie"`: die Tweedie-Devianz als Verlust l in Formel 5.

config, Z. 53 `reg_lambda` ist λ aus Formel 5, gesucht zwischen 10^{-4} und 10^{4}. Ein `gamma` steht nicht im Suchraum, es bleibt auf der Voreinstellung null.

config, Z. 47 `learning_rate` ist die Schrittweite, `n_estimators` in Z. 46 die Zahl der Bäume R.

Im Strukturstrang baut `m03_struktur.py` in Z. 90–94 einen `XGBClassifier` mit `objective="multi:softprob"`. Er rechnet für jede der vier Einsatzarten eine eigene Summe von Bäumen und macht daraus wie das Logit mit der Softmax-Funktion vier Wahrscheinlichkeiten ZUSATZ. Die Klassengewichte bekommt er als `sample_weight` beim Anpassen (Z. 151–158, 233–234).

TEIL 8

Wie gut? Die Gütemaße der Menge

> Eine einzelne Vorhersage sagt nichts über ein Modell. Erst die Fehler über alle Testzeilen eines Folds ergeben ein Gütemaß. Die Arbeit beschreibt RMSE, MAE und Bestimmtheitsmaß in Worten (S. 10). Als Formeln geschrieben sehen sie so aus:

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \qquad \mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

RMSE und MAE · in der Arbeit in Worten (S. 10)

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

Bestimmtheitsmaß · in der Arbeit in Worten (S. 10)

$y_i - \hat{y}_i$ Der Fehler einer Testzeile: wahrer Wert minus Vorhersage.

RMSE Quadriert die Fehler, mittelt sie und zieht die Wurzel. Die Wurzel bringt den Wert in die Einheit der Zielgröße zurück, also in Einsätze (S. 10). Hauptmaß der Arbeit.

MAE Mittelt die Beträge der Fehler. Große Fehler gehen nur linear ein (S. 10).

$\bar{y}$ Der Mittelwert der Testdaten. Ein Modell, das ihn vorhersagt, hätte $R^2 = 0$. Wer schlechter ist, landet unter null (S. 10).

Ein Beispiel mit vier Zeilen

Zeile	y	ŷ	Fehler	Fehler²	|Fehler|	(y − ȳ)²
1	54	56	−2	4	2	49
2	40	45	−5	25	5	441
3	120	100	+20	400	20	3.481
4	30	31	−1	1	1	961
Summe	**244**			**430**	**28**	**4.932**

Tabelle 11: Vier erfundene Zeilen ZUSATZ. Der Mittelwert der wahren Werte ist ȳ = 61.

1	RMSE	√(430 / 4) = √107,5	**10,37**
2	MAE	28 / 4	**7,00**
3	Bestimmtheitsmaß	1 − 430 / 4.932	**0,913**

Der RMSE ist nie kleiner als der MAE. Dass er hier so viel größer ist, liegt an der einen großen Abweichung von 20: Quadriert macht sie 400 der 430 aus, beim MAE zählt sie nur mit 20 von 28. Der RMSE reagiert also stärker auf einzelne große Fehler.

Die echten Werte in Fold 1

Über die 792 Testzeilen von Fold 1 in Wiederholung 0 ergeben sich diese Werte. Bernal Heights trägt zum Poisson-GLM einen Fehler von −1,74 bei, zu Ridge −5,37.

Modell	RMSE	MAE	R^2	Datei
Stufe 1: Gesamtmittelwert	57,77	35,60	−0,055	baselines_folds.csv
Stufe 2: Poisson-GLM	32,47	20,21	0,667	baselines_folds.csv
Ridge Regression	34,74	21,11	0,619	menge_folds.csv
Random Forest	46,86	30,37	0,306	menge_folds.csv
XGBoost	45,08	30,89	0,358	menge_folds.csv

Tabelle 12: Die Gütemaße auf der Einsatzzahl für Fold 1 der Wiederholung 0. Das ist einer von 50 Läufen. Die Werte in Tabelle 9 der Arbeit (S. 60) sind Mittel über alle 50 (Teil 10).

Fehler auf Rate und Anzahl: Ein Ratenfehler von 1,0 bedeutet in Seacliff mit 2.422 Einwohnern 2,4 Einsätze, in Sunset/Parkside mit 79.787 Einwohnern 79,8. Der quadratische Fehler auf der Anzahl ist deshalb der mit dem Quadrat der Einwohnerzahl gewichtete Fehler auf der Rate (S. 51).

Der Code dazu

```
65 from sklearn.metrics import (mean_absolute_error, mean_squared_error,
66                              r2_score)
67 y_true, y_pred = np.asarray(y_true, float), np.asarray(y_pred, float)
68 return {"RMSE": float(np.sqrt(mean_squared_error(y_true, y_pred))),
69         "MAE":  float(mean_absolute_error(y_true, y_pred)),
70         "R2":   float(r2_score(y_true, y_pred))}
```

vorpruefung/v1_baselines.py, Z. 65–70

Z. 68 `mean_squared_error` rechnet den Mittelwert der Fehlerquadrate, `np.sqrt` zieht die Wurzel: der RMSE.

Z. 69 `mean_absolute_error` ist der MAE.

Z. 70 `r2_score` ist das Bestimmtheitsmaß. Es nimmt als Vergleich den Mittelwert der übergebenen wahren Werte, also der Testdaten.

Für die drei Verfahren stehen dieselben drei Aufrufe in `m02_menge.py`, Z. 268–270.

TEIL 9

Wie gut? Die Gütemaße der Einsatzart

> Bei Kategorien gibt es keinen Abstand zwischen Vorhersage und Wahrheit, nur richtig oder falsch. Ausgangspunkt ist deshalb die Konfusionsmatrix: Sie zählt, wie oft welche Einsatzart als welche vorhergesagt wurde (S. 10).

Die Konfusionsmatrix des Logits in Fold 1

wahr ↓ · vorhergesagt →	Brand	Fehlalarm	Rettung	Technische Hilfe	Summe
Brand	**0**	6	0	7	13
Fehlalarm	0	**500**	35	107	642
Rettung	0	4	**1**	6	11
Technische Hilfe	0	20	40	**66**	126
Summe	**0**	**530**	**76**	**186**	**792**

Tabelle 13: Die 792 Testzeilen von Fold 1, Wiederholung 0, mit der Vorhersage des multinomialen Logits. Zeilen sind die wahre Einsatzart, Spalten die vorhergesagte. Nachgerechnet, sie trifft die gespeicherten Gütemaße genau.

Auf der Diagonale stehen die Treffer: 0 + 500 + 1 + 66 = 567. Die Accuracy ist 567/792 = 0,716. Brand wird in diesem Fold kein einziges Mal vorhergesagt, die ganze Spalte ist leer.

Formel 6

$$\mathrm{F1}_k = \frac{2\,\mathrm{P}_k\,\mathrm{R}_k}{\mathrm{P}_k + \mathrm{R}_k}, \qquad \text{Macro-F1} = \frac{1}{K}\sum_{k=1}^{K} \mathrm{F1}_k$$

Formel 6 der Arbeit · klassenweises F1-Maß und Macro-F1 (S. 10)

$$\mathrm{P}_k = \frac{\text{richtig als k vorhergesagt}}{\text{insgesamt als k vorhergesagt}} \qquad \mathrm{R}_k = \frac{\text{richtig als k vorhergesagt}}{\text{wirklich k}}$$

Präzision und Trefferquote der Klasse k, in Worten wie auf S. 11

Die Präzision liest man in der Spalte der Klasse ab, die Trefferquote in ihrer Zeile. Das F1 ist das harmonische Mittel aus beiden und fällt nur hoch aus, wenn beide hoch sind (S. 10). Sagt ein Modell eine Klasse nie vorher, ist ihre Präzision nicht definiert, und $\mathrm{F1}_k$ wird null gesetzt (S. 11).

Einsatzart	Präzision P_k	Trefferquote R_k	$F1_k$
Brand	nie vorhergesagt	0/13 = 0,000	0 (gesetzt)
Fehlalarm	500/530 = 0,943	500/642 = 0,779	0,853
Rettung	1/76 = 0,013	1/11 = 0,091	0,023
Technische Hilfe	66/186 = 0,355	66/126 = 0,524	0,423
Macro-F1 = (0 + 0,853 + 0,023 + 0,423) / 4			**0,325**

Tabelle 14: Präzision, Trefferquote und F1 je Einsatzart, abgelesen aus Tabelle 13. Beispiel Fehlalarm: F1 = 2 · 0,943 · 0,779 / (0,943 + 0,779) = 0,853.

Beide Werte stehen in `results/klassifikation/baselines_klasse.csv` (Wiederholung 0, Fold 1, Stufe 2): Macro-F1 0,325, Accuracy 0,716. Die Mehrheitsklasse kommt in denselben Zeilen auf die höhere Accuracy (0,811), aber das niedrigere Macro-F1 (0,224, Teil 2). Genau diesen Unterschied soll das Macro-F1 sichtbar machen (S. 10).

Die Macro-AUROC

Das Macro-F1 bewertet die fertige Zuordnung. Die AUROC bewertet die Wahrscheinlichkeiten davor, also wie gut sie die Zeilen der Größe nach ordnen (S. 64). Je Einsatzart wird eine Fläche gegen alle übrigen Einsatzarten berechnet, die Macro-AUROC mittelt sie mit gleichem Gewicht (S. 10).

Einsatzart	Fläche gegen die übrigen
Brand	0,747
Fehlalarm	0,880
Rettung	0,760
Technische Hilfe	0,726
Macro-AUROC, ungewichtetes Mittel	**0,778**

Tabelle 15: Die vier Flächen des Logits in Fold 1, nachgerechnet. In baselines_klasse.csv steht nur ihr Mittel 0,778.

Eine Fläche von 0,880 für Fehlalarm heißt: Zieht man zufällig einen Fehlalarm-Monat und einen Monat einer anderen Art, gibt das Modell dem Fehlalarm-Monat in 88 % der Fälle die höhere Fehlalarm-Wahrscheinlichkeit (Hand & Till 2001, S. 174) ZUSATZ. 0,5 wäre Raten. Auffällig ist Brand: Die Fläche liegt bei 0,747, das F1 bei null. Das Modell ordnet Brand-Monate also oft höher ein als andere, aber nie so hoch, dass Brand die größte der vier Wahrscheinlichkeiten wird. Ein Modell kann die Klassen gut trennen und sie dennoch schlecht zuordnen (S. 64).

Der Code dazu

```
254  "Accuracy": round(float(accuracy_score(y_te, y_hat)), 3),
255  "Macro-F1": round(float(f1_score(y_te, y_hat, average="macro",
256                                   zero_division=0)), 3),
257  "Macro-AUROC": au if np.isnan(au) else round(au, 3)})
```

vorpruefung/v1_baselines.py, Z. 254–257

```
284  return float(roc_auc_score(y_true, proba, multi_class="ovr",
285                             average="macro", labels=klassen_modell))
```

vorpruefung/v1_baselines.py, Z. 284–285

Z. 255–256 `f1_score` mit `average="macro"` ist Formel 6. `zero_division=0` setzt das F1 einer nie vorhergesagten Klasse auf null.

Z. 284–285 `roc_auc_score` mit `multi_class="ovr"` rechnet je Klasse die Fläche gegen alle übrigen, `average="macro"` mittelt ungewichtet.

Für Random Forest und XGBoost stehen dieselben Aufrufe in `m03_struktur.py`, Z. 260 und 344–345.

TEIL 10

Vom Einzellauf zum Ergebnis

> Ein Fold ergibt einen RMSE. Die Arbeit rechnet aber 50 Läufe je Verfahren: zehn Wiederholungen mit je fünf Folds. Dieser Teil zeigt, wie daraus die Zahlen in Tabelle 9 und Tabelle 10 der Arbeit werden (S. 60–61).

Zweistufig mitteln

Gemittelt wird in zwei Schritten: erst über die fünf Folds einer Wiederholung, dann über die zehn Wiederholungsmittel. Die Streuung hinter jedem Mittelwert ist die dieser zehn Werte und nicht die der 50 einzelnen Folds (S. 41–42). Für Ridge auf der Einsatzzahl sind das diese zehn Wiederholungsmittel:

29,30; 30,05; 29,35; 31,57; 28,53; 31,75; 30,49; 33,25; 32,19; 32,77

Ihr Mittel ist 30,93, ihre Standardabweichung 1,61. Das ist der Eintrag 30,93 ± 1,61 in Tabelle 9 der Arbeit (S. 60). Die Streuung über die 50 einzelnen Folds wäre mit 21,40 viel größer (menge_mittel.csv), weil sie auch enthält, dass manche Folds leicht und andere schwer sind. Die Arbeit nennt für die drei Verfahren 19,92 bis 23,81 (S. 59).

```
448  schluessel = ["zielgroesse", "verfahren"]
449  g = folds.groupby(schluessel, sort=False)
450  z = g[MASSE].mean().add_suffix("_mean")
451  z = z.join(g[MASSE].std().add_suffix("_std_folds"))
452  je_wdh = folds.groupby(schluessel + ["wiederholung"], sort=False)[MASSE].mean()
453  z = z.join(je_wdh.groupby(schluessel, sort=False).std()
454                   .add_suffix("_std_wiederholungen"))
```

modelle/m02_menge.py, Z. 448–454

Z. 450 Der Mittelwert direkt über alle 50 Läufe. Bei fünf Folds je Wiederholung ist er derselbe wie zweistufig gemittelt (S. 42).

Z. 452–453 Erst das Mittel je Wiederholung, dann die Standardabweichung über diese zehn Werte: die Streuung, die die Arbeit berichtet.

Gepaart vergleichen: Ridge gegen XGBoost

Weil beide Verfahren in jeder Wiederholung dieselben Testfolds sehen, wird je Wiederholung die Differenz gebildet (S. 59). Positiv heißt: Ridge hat den kleineren Fehler.

Wiederholung	0	1	2	3	4	5	6	7	8	9	Mittel
XGBoost	33,59	33,30	33,80	44,47	39,98	32,73	34,81	37,85	43,97	39,41	37,39
Ridge	29,30	30,05	29,35	31,57	28,53	31,75	30,49	33,25	32,19	32,77	30,93
Differenz Δ_w	**+4,29**	**+3,24**	**+4,45**	**+12,90**	**+11,45**	**+0,98**	**+4,32**	**+4,60**	**+11,77**	**+6,64**	**+6,46**

Tabelle 16: Die zehn Wiederholungsmittel des RMSE und ihre Differenz, aus `results/regression/menge_folds.csv`.

Aus den zehn Differenzen entstehen die drei Kennzahlen, die Tabelle 10 der Arbeit (S. 61) neben jeden p-Wert stellt: die mittlere Differenz, ihr 95-%-Konfidenzintervall nach der t-Verteilung und die Effektstärke d, also die mittlere Differenz geteilt durch die Standardabweichung der Differenzen (S. 19).

$$\bar{\Delta} = \frac{1}{10}\sum_{w=1}^{10} \Delta_w \qquad d = \frac{\bar{\Delta}}{s_\Delta} \qquad KI = \bar{\Delta} \pm t_{0{,}975;\,9} \cdot \frac{s_\Delta}{\sqrt{10}}$$

Mittlere Differenz, Effektstärke und Konfidenzintervall · in der Arbeit in Worten (S. 19)

1	Mittlere Differenz	Mittel der zehn Δ_w	**+6,46**
2	Standardabweichung	der zehn Δ_w	**s_Δ = 4,107**
3	Effektstärke	6,465 / 4,107	**d = 1,57**
4	Halbe Breite	2,262 · 4,107 / √10	**2,94**
5	Konfidenzintervall	6,465 ± 2,938	**[+3,53; +9,40]**

Das sind genau die Werte der Zeile „Ridge gegen XGBoost“ in Tabelle 10 der Arbeit: +6,46 [+3,53; +9,40], d = 1,57. Im Code entsteht das Intervall in `_gepaart()`, die Effektstärke rechnet `trennschaerfe.py` in Z. 44–53 aus dem Intervall zurück. Das Ergebnis ist dasselbe.

Der Wilcoxon-Test

Der Vorzeichen-Rang-Test nach Wilcoxon ordnet die Beträge der zehn Differenzen nach ihrer Größe und summiert die Ränge der positiven (S. 11). Hier sind alle zehn Differenzen positiv, alle Ränge fallen also auf die positive Seite. Wie selten das ohne echten Unterschied wäre, lässt sich direkt abzählen ZUSATZ: Gäbe es keinen Unterschied, wäre jedes Vorzeichen wie ein Münzwurf. Von den 2^{10} = 1.024 gleich wahrscheinlichen Vorzeichenmustern ist nur eines so extrem, zweiseitig zählt das umgekehrte mit. Also ist p = 2/1.024 = 0,00195.

Mit gemischten Vorzeichen rechnet der Test über die Rangsummen. Das Heft „Frage 10 und 16 einfach erklärt“ zeigt das am Vergleich Ridge gegen Poisson-GLM: Rangsumme der positiven 39, der negativen 16, p = 0,275.

Die Korrektur nach Holm

Die Vergleiche der Verfahren untereinander bilden im Mengenstrang eine Familie aus sechs Tests: drei Paarungen auf zwei Zielgrößen (S. 19). Holm prüft ihre p-Werte der Größe nach: den kleinsten gegen die strengste Schwelle, jeden weiteren gegen eine etwas mildere, bis der erste scheitert (S. 12). Gleichwertig lässt sich jeder p-Wert mit einem Faktor multiplizieren: den kleinsten mit sechs, den nächsten mit fünf und so weiter. Als angepasster p-Wert geschrieben, der direkt mit 0,05 verglichen wird, gilt dabei jeweils das bisherige Maximum, damit kein späterer Wert unter einem früheren landet ZUSATZ.

$$p_{(i)}^{Holm} = \max_{j \le i} \; \min\bigl(1,\; (m - j + 1) \cdot p_{(j)}\bigr)$$

Korrektur nach Holm als angepasster p-Wert, m Tests, p-Werte aufsteigend sortiert · Zusatz, das Verfahren steht auf S. 12

Rang	Vergleich	p	Faktor	p · Faktor	p nach Holm	Datei
1	Ridge – XGBoost, Anzahl	0,00195	6	0,0117	**0,0117**	0,0117
2	Ridge – XGBoost, Rate	0,00195	5	0,0098	**0,0117**	0,0117
3	Ridge – Random Forest, Anzahl	0,00586	4	0,0234	**0,0234**	0,0234
4	Ridge – Random Forest, Rate	0,01953	3	0,0586	**0,0586**	0,0586
5	Random Forest – XGBoost, Anzahl	0,10547	2	0,2109	**0,2109**	0,2109
6	Random Forest – XGBoost, Rate	0,43164	1	0,4316	**0,4316**	0,4316

Tabelle 17: Die Holm-Korrektur der sechs Vergleiche zwischen den Verfahren. Die vorletzte Spalte ist von Hand gerechnet, die letzte stammt aus `results/regression/vergleich.csv`. In Rang 2 gilt das Maximum aus Rang 1.

Gerundet sind das die Holm-Werte 0,012, 0,023 und 0,211 aus Tabelle 10 der Arbeit und die 0,059 auf der Rate (S. 61). Signifikant ist ein Vergleich, wenn sein Holm-Wert unter $\alpha = 0{,}05$ liegt: Das gilt für die Ränge 1 bis 3. Die Vergleiche mit der Baseline werden nicht korrigiert, weil jeder vorab einzeln festgelegt war (S. 19).

```
494 m = len(p)
495 ordnung = np.argsort(p)
496 angepasst = np.empty(m, float)
497 laufend = 0.0
498 for rang, i in enumerate(ordnung):
499     laufend = max(laufend, (m - rang) * p[i])
500     angepasst[i] = min(laufend, 1.0)
501 return angepasst
```

modelle/m02_menge.py, Z. 494–501

Z. 495 `argsort` sortiert die p-Werte aufsteigend.

Z. 499 `rang` beginnt im Code bei 0. Deshalb ist `m - rang` der Faktor aus Tabelle 17 (6, 5, …), `(m - rang) * p[i]` die Spalte p · Faktor und `max(laufend, …)` das bisherige Maximum.

Z. 500 Kein Wert darf über eins steigen.

ANHANG

Weitere Rechnungen der Arbeit

> Drei Rechnungen im Text und eine Formel gehören nicht zu den Modellen, tauchen aber in der Arbeit auf. Sie sind hier knapp nachgerechnet.

Formel 7 · der Kriminalitätsindex

$$\mathrm{Rate}(s,m) = \frac{D(s,m)}{B(s,m)}, \qquad \mathrm{Index}(s,m) = \frac{\mathrm{Rate}(s,m)}{\mathrm{Rate}(\mathrm{Stadt},m)}$$

Formel 7 der Arbeit · Kriminalitätsindex als Standortquotient (S. 34)

D(s, m) Die Delikte im Stadtteil s in den zwölf Monaten bis einschließlich m − 1. Der laufende Monat zählt nicht mit (S. 34).

B(s, m) Die Wohnbevölkerung des Stadtteils im zugeordneten Zensusjahrgang.

Rate(Stadt, m) Dieselbe Rate über alle Stadtteile zusammen: alle Delikte durch alle Einwohner (S. 34).

Ein Rechenbeispiel mit runden, erfundenen Zahlen ZUSATZ: Ein Stadtteil hat 900 Delikte in den zwölf Vormonaten und 25.000 Einwohner, also 36 Delikte je 1.000 Einwohner. Die ganze Stadt hat 51.000 Delikte bei 850.000 Einwohnern, also 60 je 1.000. Der Index ist 36/60 = 0,60. Der Stadtteil ist damit um 40 % weniger belastet als der Stadtdurchschnitt desselben Monats, ein Wert von eins hieße genau durchschnittlich (S. 35). Ins Modell geht der Logarithmus ein, hier log(0,60) = −0,51.

Echt: Bernal Heights hat im Oktober 2019 den Index 0,617, im Modell steht log(0,617) = −0,4829 (Tabelle 2). Im Code bildet `prep/s1_daten.py` in Z. 510 das Zwölfmonatsfenster ohne den laufenden Monat, Z. 522–524 die Stadtsummen und Z. 526–530 beide Raten und ihren Quotienten. Beide Raten sind dort je 1.000 Einwohner gerechnet, der Faktor kürzt sich im Index heraus. Den Logarithmus nimmt `prep/s2_datensaetze.py` in Z. 244.

Designeffekt und effektive Stichprobe

Die 4.752 Zeilen sind keine unabhängigen Beobachtungen: Jeder Stadtteil steht mit 132 einander sehr ähnlichen Monaten darin. Aus einer Intraklassenkorrelation von 0,926 folgt (S. 41):

$$1 + (132 - 1) \cdot 0{,}926 = 122{,}3 \qquad \frac{4.752}{122{,}3} = 38{,}9$$

Designeffekt 1 + (Monate je Stadtteil − 1) · Intraklassenkorrelation und effektive Stichprobe (S. 41)

Die Zeilen tragen also ungefähr so viel Information wie 39 unabhängige Beobachtungen.

Das Suchbudget

Macht ein Bereich guter Hyperparameter den Anteil v/V des Suchraums aus, trifft eine Zufallssuche mit T Ziehungen ihn mit der Wahrscheinlichkeit $1 - (1 - v/V)^T$ mindestens einmal (S. 56). Für einen Bereich von 5 %:

$$1 - (1 - v/V)^T : \qquad 1 - 0{,}95^{100} = 0{,}994 \qquad 1 - 0{,}95^{50} = 0{,}923$$

Trefferwahrscheinlichkeit der Zufallssuche (S. 56) · Code: config_modelle.py, Z. 18–21

Daher die 100 Ziehungen je Suchlauf. Die Rechnung hängt nicht davon ab, wie viele Hyperparameter ein Verfahren hat (S. 56).

Zehn Tests auf einmal

Wer zehn unabhängige Tests zum Niveau 0,05 rechnet, findet mit dieser Wahrscheinlichkeit mindestens einen Unterschied, der gar nicht besteht (S. 12):

$$1 - 0{,}95^{10} = 0{,}401$$

Wahrscheinlichkeit mindestens eines Zufallsbefunds bei zehn Tests (S. 12)

Gegen genau diese Gefahr richtet sich die Holm-Korrektur aus Teil 10.

DREI SÄTZE ZUM MITNEHMEN

Erstens: Das Poisson-GLM und die drei Verfahren schätzen eine Rate je 1.000 Einwohner und kommen erst mit der Einwohnerzahl zur Anzahl der Einsätze.

Zweitens: Die beiden Baselines und Ridge lassen sich vollständig von Hand nachrechnen, der Random Forest über seine Bäume, XGBoost über die Summe seiner Bäume, aber nicht überall auf die letzte Stelle.

Drittens: Über die Güte entscheidet nie eine einzelne Zeile, sondern der Fehler über alle Testzeilen, gemittelt über zehn Wiederholungen und gepaart getestet.